

python.control-flow#

<https://pytorch.org/docs/stable/generated/exportdb/python.control-flow.html>

Description:

Tags: torch.dynamic-shape, python.control-flow Support Level:
SUPPORTED Original source code:

Code Examples

```

# mypy: allow-untyped-defs
import torch

class DynamicShapeIfGuard(torch.nn.Module):
    """
    `if` statement with backed dynamic shape predicate will be
    specialized into
    one particular branch and generate a guard. However, export
    will fail if the
    the dimension is marked as dynamic shape from higher level
    API.
    """

    def forward(self, x):
        if x.shape[0] == 3:
            return x.cos()

        return x.sin()

example_args = (torch.randn(3, 2, 2),)
tags = {"torch.dynamic-shape", "python.control-flow"}
model = DynamicShapeIfGuard()

torch.export.export(model, example_args)

```

ExportedProgram:

```
class GraphModule(torch.nn.Module):  
    def forward(self, x: "f32[3, 2, 2]"):   
        cos: "f32[3, 2, 2]" =   
torch.ops.aten.cos.default(x);  x = None  
        return (cos,)
```

Graph signature:

```
# inputs  
x: USER_INPUT
```

```
# outputs  
cos: USER_OUTPUT
```

Range constraints: {}

```
# mypy: allow-untyped-defs

import torch

class ListUnpack(torch.nn.Module):
    """
    Lists are treated as static construct, therefore unpacking
    should be
    erased after tracing.
    """

    def forward(self, args: list[torch.Tensor]):
        """
        Lists are treated as static construct, therefore
        unpacking should be
        erased after tracing.
        """
        x, *y = args
        return x + y[0]

example_args = ([torch.randn(3, 2), torch.tensor(4),
torch.tensor(5)],)
tags = {"python.control-flow", "python.data-structure"}
model = ListUnpack()

torch.export.export(model, example_args)
```

ExportedProgram:

```
class GraphModule(torch.nn.Module):
    def forward(self, args_0: "f32[3, 2]", args_1: "i64[]",
args_2: "i64[]"):
        add: "f32[3, 2]" =
torch.ops.aten.add.Tensor(args_0, args_1);  args_0 = args_1 =
None
        return (add,)
```

Graph signature:

```
# inputs
args_0: USER_INPUT
args_1: USER_INPUT
args_2: USER_INPUT

# outputs
add: USER_OUTPUT
```

Range constraints: {}

```
# mypy: allow-untyped-defs
import torch

class StaticForLoop(torch.nn.Module):
    """
    A for loop with constant number of iterations should be
    unrolled in the exported graph.
    """

    def forward(self, x):
        # constant
        ret = [i + x for i in range(10)]
        return ret

example_args = (torch.randn(3, 2),)
tags = {"python.control-flow"}
model = StaticForLoop()

torch.export.export(model, example_args)
```

ExportedProgram:

```
class GraphModule(torch.nn.Module):
    def forward(self, x: "f32[3, 2]"):
        add: "f32[3, 2]" = torch.ops.aten.add.Tensor(x,
0)
            add_1: "f32[3, 2]" = torch.ops.aten.add.Tensor(x, 1)
            add_2: "f32[3, 2]" = torch.ops.aten.add.Tensor(x, 2)
            add_3: "f32[3, 2]" = torch.ops.aten.add.Tensor(x, 3)
            add_4: "f32[3, 2]" = torch.ops.aten.add.Tensor(x, 4)
            add_5: "f32[3, 2]" = torch.ops.aten.add.Tensor(x, 5)
            add_6: "f32[3, 2]" = torch.ops.aten.add.Tensor(x, 6)
            add_7: "f32[3, 2]" = torch.ops.aten.add.Tensor(x, 7)
            add_8: "f32[3, 2]" = torch.ops.aten.add.Tensor(x, 8)
            add_9: "f32[3, 2]" = torch.ops.aten.add.Tensor(x,
9); x = None
        return (add, add_1, add_2, add_3, add_4, add_5,
add_6, add_7, add_8, add_9)
```

Graph signature:

```
# inputs
x: USER_INPUT

# outputs
add: USER_OUTPUT
add_1: USER_OUTPUT
add_2: USER_OUTPUT
add_3: USER_OUTPUT
add_4: USER_OUTPUT
add_5: USER_OUTPUT
add_6: USER_OUTPUT
add_7: USER_OUTPUT
add_8: USER_OUTPUT
add_9: USER_OUTPUT
```

Range constraints: {}

```
# mypy: allow-untyped-defs
import torch

class StaticIf(torch.nn.Module):
    """
    `if` statement with static predicate value should be traced
    through with the
    taken branch.
    """

    def forward(self, x):
        if len(x.shape) == 3:
            return x + torch.ones(1, 1, 1)

        return x

example_args = (torch.randn(3, 2, 2),)
tags = {"python.control-flow"}
model = StaticIf()

torch.export.export(model, example_args)
```


ExportedProgram:

```
class GraphModule(torch.nn.Module):
    def forward(self, x: "f32[3, 2, 2]"):
        ones: "f32[1, 1, 1]" =
torch.ops.aten.ones.default([1, 1, 1], device =
device(type='cpu'), pin_memory = False)
        add: "f32[3, 2, 2]" = torch.ops.aten.add.Tensor(x,
ones);  x = ones = None
        return (add,)
```

Graph signature:

```
# inputs
x: USER_INPUT
```

```
# outputs
add: USER_OUTPUT
```

Range constraints: {}

Notes & Warnings

NOTE

Note Tags: torch.dynamic-shape, python.control-flow Support Level: SUPPORTED

NOTE

Note Tags: python.data-structure, python.control-flow Support Level: SUPPORTED

NOTE

Note Tags: python.control-flow Support Level: SUPPORTED

NOTE

Note Tags: python.control-flow Support Level: SUPPORTED

Detailed Documentation

dynamic_shape_if_guard#

Original source code:

list_unpack#

Original source code:

static_for_loop#

Original source code:

static_if#

Original source code:

Tags: torch.dynamic-shape, python.control-flow

Support Level: SUPPORTED

Original source code:

Tags: python.data-structure, python.control-flow

Support Level: SUPPORTED

Original source code:

Tags: python.control-flow

Support Level: SUPPORTED

Original source code:

Tags: python.control-flow

Support Level: SUPPORTED

Original source code:

